PART 06 — DATABASE SCHEMA & RELATIONAL ERD

Database Engine: PostgreSQL 16+

ORM: Prisma ORM

Precision Standard: Fixed-point 4 decimal places (`NUMERIC(19,4)`)

Isolation Architecture: Composite Tenant Foreign Keys (`organization_id, id`)

1. Relational Database Overview (Non-Technical Explanation)

Think of blynt's database as an organized filing cabinet system for a corporate building:

- **The Master Cabinet (`organizations`):** Every drawer represents a distinct company. All invoices, accounts, customer records, and tax filings belong to exactly one drawer.
- **The File Folders (`accounts` - Chart of Accounts):** Inside a company's drawer are 46 standard UK folders. For example, Folder 1000 is for Cash in Bank, Folder 1100 is for Customer Debtors, Folder 4000 is for Sales Revenue.
- **The Double-Entry Index Cards (`journal_entries` & `journal_lines`):** Every financial movement is written on an index card that has at least two lines. If money leaves Folder 1000, it must enter another folder.
- **The Contact Cards (`contacts`):** Cards representing customers you sell to or suppliers you buy from.
- **The Tamper-Proof Black Box (`audit_logs`):** A special log where every pen stroke made on any folder is photographed and timestamped. It is physically impossible to erase any entry from this box.

2. Authoritative Relational Entities & Attributes

2.1 Entity: `organizations` (The Multi-Tenant Workspace)

Stores company profile, registration numbers, and jurisdiction controls.

Column	Data Type	Nullable	Description & Constraints
`id`	`VARCHAR(36)`	No	Primary Key (`cuid` / UUID)
`legal_name`	`VARCHAR(255)`	No	Registered business entity name (e.g., "Demo Consulting Ltd")
`slug`	`VARCHAR(100)`	No	Unique URL-safe identifier per organisation
`company_number`	`VARCHAR(50)`	Yes	UK Companies House 8-character registration number
`vat_number`	`VARCHAR(50)`	Yes	UK HMRC 9-digit VAT registration number (e.g., "GB123456789")

`base_currency`	`VARCHAR(3)`	No	Default currency code (Default: `GBP`)
`status`	`OrganizationStatus`	No	Enum: `ACTIVE`, `SUSPENDED`, `ARCHIVED`
`created_at`	`TIMESTAMPTZ`	No	System creation timestamp (`now()`)
`updated_at`	`TIMESTAMPTZ`	No	Auto-updating modification timestamp

2.2 Entity: `accounts` (The Nominal General Ledger)

Represents the UK Chart of Accounts nominal codes.

Column	Data Type	Nullable	Description & Constraints
`id`	`VARCHAR(36)`	No	Primary Key
`organization_id`	`VARCHAR(36)`	No	Foreign Key -> `organizations.id`
`code`	`VARCHAR(20)`	No	Nominal account code (e.g., "1000", "1100", "4000")
`name`	`VARCHAR(255)`	No	Human-readable title (e.g., "Accounts Receivable Control")
`type`	`AccountType`	No	Enum: `ASSET`, `LIABILITY`, `EQUITY`, `REVENUE`, `EXPENSE`
`subtype`	`AccountSubtype`	No	Enum: `BANK`, `ACCOUNTS_RECEIVABLE`, `SALES`, etc.
`normal_balance`	`BalanceType`	No	Enum: `DEBIT` or `CREDIT`
`is_control_account`	`BOOLEAN`	No	If `true`, direct manual journal entries are strictly blocked
`allow_manual_posting`	`BOOLEAN`	No	If `false`, cannot be chosen in manual journal line dropdown
`status`	`AccountStatus`	No	Enum: `ACTIVE`, `ARCHIVED`

- **Unique Composite Key:** `UNIQUE (organization_id, code)` — Ensures code 1000 is unique within Company A, but allows Company B to have its own code 1000.

2.3 Entities: `journal_entries` & `journal_lines` (The Double-Entry Core)

`journal_entries` (Header Record)

Column	Data Type	Nullable	Description & Constraints
`id`	`VARCHAR(36)`	No	Primary Key
`organization_id`	`VARCHAR(36)`	No	Foreign Key -> `organizations.id`
`entry_number`	`VARCHAR(50)`	No	Atomic sequential number: `JE-YYYY-XXXXXX` (Unique per org)
`entry_date`	`DATE`	No	Calendar date of transaction
`status`	`JournalStatus`	No	Enum: `DRAFT`, `VALIDATED`, `POSTED`, `REVERSED`
`source_type`	`JournalSource`	No	Enum: `MANUAL`, `SALES_INVOICE`, `PURCHASE_BILL`, `PAYMENT`
`description`	`TEXT`	Yes	Accounting memo / description

`journal_lines` (Debit & Credit Detail Lines)

Column	Data Type	Nullable	Description & Constraints
`id`	`VARCHAR(36)`	No	Primary Key
`organization_id`	`VARCHAR(36)`	No	Foreign Key -> `organizations.id` (Composite isolation)
`journal_entry_id`	`VARCHAR(36)`	No	Foreign Key -> `journal_entries.id`
`account_id`	`VARCHAR(36)`	No	Foreign Key -> `accounts.id`
`contact_id`	`VARCHAR(36)`	Yes	Optional Foreign Key -> `contacts.id` (AR/AP subledger tag)

`debit`	`NUMERIC(19,4)`	No	Debit amount (Default: `0.0000`)
`credit`	`NUMERIC(19,4)`	No	Credit amount (Default: `0.0000`)
`description`	`TEXT`	Yes	Line-level item narrative

- **Database Constraints Enforced on `journal_lines`:**
- `CHECK (debit >= 0 AND credit >= 0)` — Negative numbers are physically rejected.
- `CHECK (NOT (debit = 0 AND credit = 0))` — Zero-amount lines are rejected.
- `CHECK (NOT (debit > 0 AND credit > 0))` — A single line cannot have both a debit and a credit.
- `FOREIGN KEY (organization_id, contact_id) REFERENCES contacts(organization_id, id)` — Physically prevents linking a journal line in Organisation A to a Contact in Organisation B.

2.4 Entities: `financial_years` & `accounting_periods`

- **`financial_years`:**
- Enforces PostgreSQL temporal exclusion:

`sql

ALTER TABLE financial_years

ADD CONSTRAINT financial_years_no_overlap

EXCLUDE USING gist (

organization_id WITH =,

daterange(start_date, end_date, '[]') WITH &&

);

- **`accounting_periods`:**
- Contains `period_number` (1 to 12), `start_date`, `end_date`, and `status` (`OPEN`, `SOFT_LOCKED`, `HARD_LOCKED`).

2.5 Entity: `audit_logs` (The Immutable Black Box)

Stores every security and financial modification across the platform.

`sql

-- Trigger preventing any modification or deletion of historical audit logs

CREATE OR REPLACE FUNCTION prevent_audit_log_mutation()

RETURNS TRIGGER AS $$

BEGIN

RAISE EXCEPTION 'Statutory Compliance Violation: audit_logs are immutable and cannot be updated or deleted';

END;

$$ LANGUAGE plpgsql;

```
CREATE TRIGGER trg_audit_logs_immutable
BEFORE UPDATE OR DELETE ON audit_logs
FOR EACH ROW EXECUTE FUNCTION prevent_audit_log_mutation();
```